Extracting Implicit Procedural Knowledge: Partial-Order Structure in LLM Representations

Mark Martirosian, Sveta Paster Shainkopf, Reuth Mirsky

Department of Computer Science, Tufts University
Medford, MA, USA
{Svetlana.Paster, Reuth.Mirsky}@tufts.edu, mmartiro22@gmail.com

Abstract

Collaborative robots need explicit representations of human plans that accommodate behavioral variability. Partially ordered plans (POPs) provide this flexibility by encoding necessary action-ordering constraints while leaving independent actions unordered, but constructing them typically requires domain knowledge or task-specific data. LLMs may instead contain implicit procedural models learned from human-generated text, raising a central question: can we elicit these internal representations to recover external, symbolic models of human plans?

We investigate whether pairwise ordering relations are linearly accessible in the hidden states of Mistral-7B-Instruct-v0.1. Using human-annotated POPs from ProScript, we train two linear probes to identify whether one action in a model of a human plan should directly precede another and whether any direct ordering constraint exists between a pair. Both properties are significantly decodable, achieving best-layer development AUCs of 0.900 and 0.869. On held-out plans, the probes also outperform the LLM's explicit judgments of the same relations by 0.16 and 0.11 F1, respectively. These findings provide initial evidence that procedural relations relevant to explicit human-plan representations are linearly accessible in LLM hidden states, opening a path from learned foundation-model representations to flexible, interpretable task models for collaborative robots.

Introduction and Related Work

Effective human–robot collaboration requires robots to reason not only about what a person is doing, but also about how flexibly a task can be carried out. Everyday procedures such as preparing a meal, assembling furniture, or carrying out a household task rarely admit only one valid execution sequence. People naturally reorder independent actions according to context, preference, or convenience, as illustrated by the sandwich-making example in Figure 1. A collaborative robot that assumes a single fixed sequence may therefore interpret a perfectly reasonable deviation as an error, fail to recognize the person's progress, or miss opportunities to assist. Representing which actions are constrained and which can be flexibly reordered is consequently important for robots that must adapt to human behavior rather than require humans to follow a predefined script.

Partially ordered plans naturally represent this flexibility (Kautz and Allen 1986; Sacerdoti 1975). A procedural task

Copyright © 2027, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

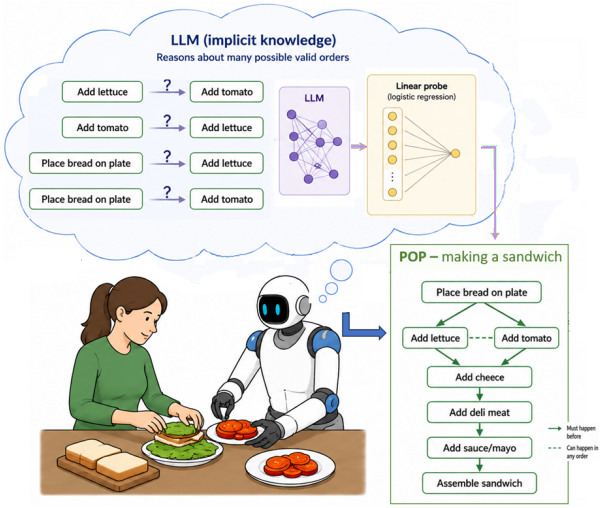


Figure 1: Sandwich-making example illustrating that pairwise action-ordering relations may be extracted from LLM hidden representations.

can be represented as a directed acyclic graph (DAG) over its actions, where ordering constraints capture which actions must precede others, while incomparable actions can occur in different orders. Unlike a total-order representation, which commits to one execution sequence, a partial-order representation compactly captures multiple valid executions. The central challenge, however, is obtaining this structure. Manually specifying ordering constraints requires domain knowledge and does not scale across the diversity of everyday procedures.

Previous work has approached this problem by learning or generating procedural structure from observable data (Sakaguchi et al. 2021; Seminara, Farinella, and Furnari 2024). ProScript (Sakaguchi et al. 2021) for example, represents everyday procedures as partially ordered scripts and learns dependency structure from annotated data. LLMs offer a potential alternative source for constructing such representations: through training on large human-generated corpora, they are exposed to descriptions of diverse activities and procedures. This scale of procedural data that LLMs are trained on raises the question of whether they capture the action

relations needed to represent these procedures. TaskLAMA (Yuan et al. 2023) shows that language models can generate plausible steps for complex tasks while remaining substantially weaker at predicting pairwise temporal dependencies. Thus, extracting the ordering relations needed for partial-order structure through generation remains challenging.

Rather than asking an LLM to *generate* the ordering structure it has learned, we can ask whether such structure is represented in its internal activations. Probing tests this possibility by determining whether a property can be decoded from frozen hidden representations. In particular, a linear probe tests whether that property is accessible through a linear decision boundary in the model's representation space. Successful decoding does not establish that the model explicitly reasons using the recovered relation, but provides evidence that the information *is linearly represented in its hidden states*.

Recent work provides encouraging evidence that LLM hidden states encode structured information of this kind. Hidden representations have been shown to support linear readout of evolving entity states (Li, Nye, and Andreas 2021) and latent world-state propositions (Feng, Russell, and Steinhardt 2024). More directly related to structural representations, Zhong, He, and Mesgarani (2026) find that frozen LLM activations linearly encode properties of reasoning DAGs, including node depth, pairwise distance, and adjacency, with strong signals in intermediate layers. Other work finds evidence for hierarchical structure (Dawes et al. 2026) and planning-relevant information, including valid continuations that are not ultimately generated (Ustaomeroglu et al. 2025). Together, these findings suggest that hidden states can encode relational, graph-like, and planning-relevant information beyond what is directly observable in generated output.

However, these studies do not address the particular relations required by procedural partial-order plans. Existing structural probes primarily examine properties such as adjacency, distance, or hierarchy in reasoning structures, rather than ordering relations between actions in everyday procedures. Conversely, work on procedural knowledge has largely focused on predicting or generating explicit task structures rather than examining their representation within LLM hidden states. To our knowledge, it therefore remains unclear whether procedural action-ordering relations themselves have a linearly accessible representation. In particular, recovering partial-order structure requires determining whether a specified precedence relation between two actions holds, and whether the actions are constrained in their relative order or are flexibly reorderable. This gap between procedural representations and what is known about LLM hidden representations motivates our central research question:

RQ: Are pairwise ordering relations between actions in procedural tasks linearly represented in the hidden states of a large language model?

We investigate this question using Mistral-7B-Instruct-v0.1 and human-annotated partial-order plans from ProScript. We explore the research question through two complementary linear probing tasks: a *direct-precedence entailment probe*, which tests whether a specified direct precedence relation $a_i \prec a_j$ holds for an ordered pair of actions, and an *ordering-constraint detection probe*, which tests whether any direct ordering constraint exists between two actions or whether they are flexible with respect to one another. We additionally compare the probes with the model's explicit output on the same relations. If these relations can be recovered from LLM representations, they could provide the ordering constraints needed to construct partial-order task models, allowing a robot to distinguish between violations of required dependencies and valid reorderings of independent human actions.

Technical Approach

We train two separate linear probes on the hidden states of Mistral-7B-Instruct-v0.1. Each probe uses a dedicated prompt; the full prompts are provided in the Appendix. The probes form a factorized representation of pairwise relations in a procedural partial order.

Probe 1: Pairwise direct-precedence entailment. Given an ordered pair of actions, this probe predicts whether the procedure directly entails that $a_i \prec a_j$. Thus, Probe 1 tests whether a directed precedence relation is linearly decodable from the representation.

Probe 2: Pairwise ordering-constraint detection. Given a pair of actions, this probe predicts whether the procedure imposes any precedence constraint between them. A positive prediction indicates that the actions are comparable: either $a_i \prec a_j$ or $a_j \prec a_i$. A negative prediction indicates that the actions are incomparable, denoted by $a_i \parallel a_j$.

For example, in the sandwich-making process in Figure 1, "put bread on plate" and "add lettuce" have a precedence relation, whereas "add lettuce" and "add tomato" may be incomparable. Probe 1 tests the direction of the relation, while Probe 2 checks for any ordering constraint.

The probes are trained and evaluated independently because they represent different factors of a partial-order relation. Probe 1 asks whether one specified direction holds, whereas Probe 2 asks whether any ordering constraint exists. Together, their predictions can be decoded into the following ternary relation:

$$\{a_i \prec a_j,\ a_j \prec a_i,\ a_i \parallel a_j\}$$

Inconsistent joint predictions, such as a precedence prediction together with an incomparability prediction, are treated as constraint violations.

Linear probing is used as a minimal-assumption test of whether procedural relations are linearly decodable from the representation. A successful linear probe shows that the target information is accessible through a simple readout, without requiring the probe to perform substantial nonlinear computation. The linear probe technique offers a cleaner but more limited representation than a higher-capacity probe.

The method does not assume that hidden states are intrinsically more reliable than textual outputs. Rather, it tests whether they contain decodable procedural information that is not consistently expressed in a sampled response. This question is related to prior work probing graph structure and latent state information from frozen language-model repre-

sentations (Zhong, He, and Mesgarani 2026; Feng, Russell, and Steinhardt 2024).

Same-depth sibling pairs for Probe 2

Probe 2 is trained using same-depth incomparable pairs as negative examples. Specifically, these are pairs of actions at the same topological depth that share a parent and have no path between them. For example, in a plan for *making a sandwich*, the steps *add lettuce* and *add tomato* may both depend on *place bread on plate* but have no ordering constraint between them; they are incomparable siblings at the same depth. These pairs provide structurally matched hard negatives for an unnecessary ordering constraint, because the actions are plausible siblings within the same local part of the procedure.

This restriction produces a targeted negative class rather than the full distribution of incomparable pairs. Consequently, we evaluate under two conditions: a same-depth condition, which matches the probe's training data, and a cross-depth condition, which uses incomparable pairs from different branches of the DAG and measures the probe's structural generalization to a more challenging task. A combined condition pools both negative types and reflects the overall performance across the full set of spurious pairs.

For each probe, we sweep candidate layers within a narrow mid-network window, layers 15-18, and chose the best performing layer (Table 3 in Appendix). A preliminary full-depth sweep suggested that procedural ordering information was strongest in this region. We therefore restrict the main sweep to this window to reduce computation; this layer choice should be treated as a preliminary empirical finding.

We use label-permutation testing to assess whether probe performance exceeds chance. We permute labels and refit the probe to produce a null distribution of cross-validated AUC values. The observed AUC is then compared with this distribution. This test shows above-chance decodability under the chosen data split and does not by itself establish that the probed information is causally used by the language model (Table 4 in Appendix). We use label permutation testing because it verifies that the probe's high AUC results from genuine signal, not an artifact of the training procedure.

Experiments & Results

We use Mistral-7B-Instruct-v0.1 for our evaluation. We chose this model because it is an open-weight, instruction-tuned model of moderate scale with full hidden-state access, consistent with prior work probing procedural and factual knowledge in transformer representations; open weights are a requirement for this method, since probing requires access to internal activations that closed API models do not expose.

We evaluate on ProScript (Sakaguchi et al. 2021). This dataset provides human-annotated procedural plans with explicit partial-ordering structure (plans that involve certain step pairs with a required order, while others can be interchanged without affecting the overall process) rather than purely linear sequences. This structure is exactly what both probing tasks require: Probe 1 needs real examples of required orderings to recover, and Probe 2 needs real examples of steps that are incomparable by design, not merely unordered due to missing annotation. ProScript's official train/dev/test splits also let us separate layer selection, threshold locking, and final evaluation without any of the three touching the same plans.

Both probes are implemented as a two-step pipeline: the hidden state vectors described below are first standardized using `StandardScaler` (zero mean, unit variance across the training set), and then passed to a `LogisticRegression` classifier trained with L2 regularization. The scaler's parameters are fit on the training split only and applied without refitting to the dev and test splits. Logistic regression is a linear classifier used in probing studies, as it introduces no nonlinearity beyond the sigmoid link, demonstrating that information is linearly decodable from the hidden state.

For each pair of actions that we wish to evaluate, we construct the prompt mentioned in the Appendix and perform a single forward pass through the model (this forward pass is used for both the probe and LLM baseline). We extract the hidden state of the final prompt token at the selected layer, which gives us a d=4096-dimensional vector. This vector is the sole input to the linear classifier.

The ground truth for this comparison is the ProScript DAG annotation for each pair a_i, a_j (the same $y \in \{0, 1\}$ label the probe is trained and evaluated on). The LLM baseline score p_{yes} is then thresholded against this label using the same held-out test split and locked dev-set threshold as the probe. Rather than decoding a full response, we extract the next-token log-probabilities at the answer position and compute $p_{\text{yes}} = \text{softmax}(z)_{\text{yes}}$, normalized over the vocabulary; in other words, the raw normalized probability assigned to the "yes" token as an answer. For Probe 2, this probability is inverted to $1 - p_{\text{yes}}$ before threshold evaluation, due to the different framing of the prompt, as described in the Appendix.

Overall, both probes outperform the LLM output baseline on held-out plans. For direct-precedence entailment (Probe 1), the probe achieves an F1 of 0.77 compared with 0.61 for the LLM output baseline (Table 1). For ordering-constraint detection (Probe 2), the probe achieves a combined F1 of 0.81 compared with 0.70 for the LLM baseline and maintains an advantage in both the same-depth and cross-depth conditions (Table 2).

Statistical Analysis

We report F1, precision, recall, specificity [1], and Matthews Correlation Coefficient (MCC); Probe–LLM differences are evaluated using McNemar's exact test, with ΔMCC and Cohen's h as effect-size measures. Thresholds are selected on the development set, and 95% F1 confidence intervals use 2,000 plan-level bootstrap resamples. Further statistical details are provided in the Appendix.

[1] Specificity measures the proportion of negative examples correctly identified, $\text{TN}/(\text{TN} + \text{FP})$. For Probe 1, negatives are ordered pairs for which the queried direct precedence relation $a_i \prec a_j$ does not hold. For Probe 2, negatives are incomparable action pairs ($a_i \parallel a_j$); thus, specificity measures how well the probe recognizes actions that can be flexibly reordered.

System	F1	P	R	Specificity
Probe	**0.77**	0.75	0.8	**79.4%**
LLM output	0.61	0.44	0.98	1.5%

Table 1: Probe 1 (pairwise precedence entailment): held-out test set, 31,680 pairs, 43.8% positive. McNemar $p < 0.001$ (b=14144, c=2779); $\Delta\text{MCC} = +0.599$; Cohen's $h = 1.96$ (large) for the specificity gap.

Condition	System	F1	P	R	Specificity
Combined	Probe	**0.81**	0.80	0.82	**68.3%**
Combined	LLM	0.70	0.63	0.79	28.0%
Same-depth	Probe	**0.85**	0.87	0.82	**73.1%**
Same-depth	LLM	0.75	0.72	0.79	32.3%
Cross-depth	Probe	**0.86**	0.91	0.82	**56.9%**
Cross-depth	LLM	0.81	0.83	0.79	17.9%

Table 2: Probe 2 (Pairwise ordering-constraint detection): 817 multi-ordering test plans, 10,095 pairs, 60.9% positive. Combined: McNemar $p < 0.001$ (b=2912, c=1102); $\Delta\text{MCC} = +0.433$; Cohen's $h = 0.83$ (large) for the specificity gap. Cross-depth Cohen's $h = 0.83$ (large).

Discussion

Our results provide evidence that both precedence and ordering-constraint information are linearly decodable from Mistral-7B-Instruct hidden representations. For Probe 1, the performance difference is driven largely by the LLM output baseline defaulting to "yes": its specificity is only 1.5%, compared with 79.4% for the probe. This represents a large specificity difference (Cohen's $h = 1.96$) and indicates that the LLM baseline has difficulty distinguishing true direct-precedence relations from non-edges, despite its high recall.

For Probe 2, interpretation of the comparison requires considering the prompt design. We originally used a different prompt (Prompt 2B in the Appendix) that produced substantially poorer specificity. In the cross-depth condition, the strong class imbalance (6,147 positive pairs versus 1,182 negative pairs), together with the LLM's tendency to predict "yes," inflated F1 through majority-class recall. The original prompt also framed the question in terms of whether an ordering was "necessary," which may have biased the model toward a positive response. Because of the resulting low specificity, we tested a flexibility-framed prompt (Prompt 2 in the Appendix) intended to make the distinction between constrained and incomparable pairs more explicit. Importantly, this prompt was introduced after examining the held-out test results, and the corresponding baseline comparison should therefore be interpreted with this limitation in mind. With the flexibility-framed prompt, LLM specificity in the cross-depth condition increased from 0.5% to 17.9%.

Probe 2 also shows a generalization gap: specificity decreases from 73.1% on same-depth pairs to 56.9% on cross-depth pairs, consistent with the probe being trained only on same-depth negative examples. Nevertheless, the probe retains substantially higher cross-depth specificity than the LLM baseline (56.9% versus 17.9%), corresponding to a large specificity difference (Cohen's $h = 0.83$). The cross-depth condition also tests whether the learned decision boundary transfers beyond the sibling structure used to construct the negative training examples. This is relevant to partial-order representations, where incomparable actions may occur across different branches and need not share a parent or topological depth.

Finally, the probe's advantage over the LLM output baseline should be interpreted cautiously. The probe receives supervised training on labeled ProScript examples, while the LLM baseline receives no comparable task-specific supervision. The performance gap therefore demonstrates the utility of a supervised linear readout, but does not by itself establish that the model possesses procedural ordering information that it is unable to express through its output.

Conclusion and Future Work

The two relations examined in this work correspond to basic decisions required when constructing a partial-order task model: whether two actions are constrained in their relative order and, when they are, which precedence relation holds. Our results provide initial evidence that information relevant to both decisions is linearly accessible in Mistral-7B-Instruct hidden representations. This does not yet constitute automatic POP construction: the present experiments evaluate pairwise relations independently and rely on supervised probes. However, they suggest a possible interface between learned language-model representations and explicit symbolic task models, where recovered pairwise relations could eventually be assembled and validated as a partial-order graph.

Several steps remain before these pairwise relations can be used to construct complete partial-order plans. Future work should test whether the recovered structure generalizes across procedural datasets beyond ProScript and across LLMs of different scales and training regimes. It should also extend probing beyond directly connected action pairs to transitive dependencies, testing whether hidden states represent not only local ordering relations but the broader logical closure of a plan's dependency structure. Finally, to ensure a fairer comparison between the probe and the LLM output, future work should evaluate the LLM with few-shot prompting instead of zero-shot; this would reduce the supervision asymmetry between them, and clarify whether the probe's advantage was due to a lack of supervision or genuine signal. Together, these directions will help determine whether implicit LLM representations can be reliably translated into explicit, interpretable partial-order plans suitable for robotic reasoning and collaboration.

Acknowledgement

This work has taken place in the Goal Optimization using Learning and Decision-Making (GOLD) lab at Tufts University. GOLD research is supported in part by NSF (OIA-2535195) and GE Vernova.

References

Dawes, C.; Sharma, A.; Lagos, A. I.; and Raval, S. 2026. H-Probes: Extracting Hierarchical Structures From Latent Representations of Language Models.

Feng, J.; Russell, S.; and Steinhardt, J. 2024. Monitoring Latent World States in Language Models with Propositional Probes. *ArXiv*.

Kautz, H. A.; and Allen, J. F. 1986. Generalized Plan Recognition. In *AAAI Conference on Artificial Intelligence*.

Li, B. Z.; Nye, M.; and Andreas, J. 2021. Implicit Representations of Meaning in Neural Language Models.

Sacerdoti, E. D. 1975. The Nonlinear Nature of Plans. In *International Joint Conference on Artificial Intelligence*.

Sakaguchi, K.; Bhagavatula, C.; Le Bras, R.; Tandon, N.; and Clark, P. 2021. proScript: Partially Ordered Scripts Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2138–2149. Association for Computational Linguistics.

Seminara, L.; Farinella, G. M.; and Furnari, A. 2024. Differentiable Task Graph Learning: Procedural Activity Representation and Online Mistake Detection from Egocentric Videos. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Ustaomeroglu, M.; Askin, B.; Joshi, G.; Joe-Wong, C.; and Qu, G. 2025. Internal Planning in Language Models: Characterizing Horizon and Branch Awareness.

Yuan, Q.; Kazemi, M.; Xu, X.; Noble, I.; Imbrasaite, V.; and Ramachandran, D. 2023. TaskLAMA: Probing the Complex Task Understanding of Language Models. *ArXiv*.

Zhong, T.; He, L.; and Mesgarani, N. 2026. From Chains to DAGs: Probing the Graph Structure of Reasoning in LLMs. *ArXiv*.

Appendix

This appendix provides the full prompts used in each stage of the pipeline, as well as the exact metrics regarding layer selection.

Layer choosing

Probe	Layer	Dev AUC	Dev macro-F1
Probe 1 (direct precedence)	16	0.900	0.912
Probe 2 (necessity)	17	0.869	0.857

Table 3: Layer selection per probe (dev set).

Probe	Layer	Real CV AUC	Permuted mean	p
Probe 1	16	0.836	0.499	0.032
Probe 2	17	0.841	0.500	0.032

Table 4: Permutation test (30 permutations, 3-fold GroupK-Fold CV on a $\leq$5,000-row training subsample). $p = 0.032$ is the floor value at 30 permutations; 0/30 permuted runs matched the real AUC.

Prompts

Prompt 1: Ordering Direction

```
[INST] You are judging a temporal
dependency between two actions in a task.
Task: {goal}
Action A: {a}
Action B: {b}
Question: Must Action A happen
before Action B? Answer yes or no.
Answer: [/INST]
```

The hidden state is extracted at the answer token position. A "yes" response corresponds to the positive class (y=1, real directed edge); "no" corresponds to the negative class (y=0, reversed or incomparable pair). The LLM baseline uses the normalized probability assigned to the "yes" token at this position.

Prompt 2B: Ordering Necessity

```
[INST] You are judging whether a
specific ordering constraint is
required.
Task: {goal}
Step A: {a}
Step B: {b}
Question: Is it strictly necessary
for A to happen before B, or could
they happen in either order without
affecting the task?
Answer yes (necessary) or no (flexible).
Answer: [/INST]
```

Prompt 2: Ordering Flexibility

```
[INST] You are examining two actions in a
task to see if they can be
done in any order.
Task: {goal}
Step A: {a}
Step B: {b}
Question: Could A and B happen in either
order without affecting the
task, or is one order strictly required?
Answer yes (flexible) or no (required).
Answer: [/INST]
```

The hidden state is extracted at the answer token position. Because this prompt leads with the flexible possibility, a "yes" response corresponds to the negative class (y=0, spurious/incomparable pair) and "no" corresponds to the positive class (y=1, real required edge). The LLM baseline probability p_{yes} is therefore inverted to $1 - p_{\text{yes}}$ before threshold evaluation, so that higher values consistently predict the positive class across both probes.

Note: the original Probe 2 prompt used a necessity-leading framing ("Is it strictly necessary for A to happen before B?"), which produced near-blanket "yes" predictions from the LLM (specificity 0.9%, recall 0.996). The current flexibility-leading framing was adopted after ablation confirmed the original framing introduced a leading bias; see the Experiments & Results section for details.

Statistical Analysis Details

We use McNemar's exact binomial test because the probe and LLM baseline are evaluated on the same held-out examples. The test evaluates the asymmetry between cases in which the probe is correct and the LLM is incorrect and cases in which the LLM is correct and the probe is incorrect.

We additionally report the Matthews Correlation Coefficient (MCC) for both systems and their difference ΔMCC. MCC accounts for all four cells of the confusion matrix and is therefore informative under class imbalance.

Cohen's h is reported for the specificity difference between the probe and LLM baseline. It provides a standardized effect size for the difference between two proportions; $|h| = 0.2$, 0.5, and 0.8 correspond to small, medium, and large effects, respectively.

All thresholds are selected on the development set and are not re-optimized on the test set. Bootstrap 95% confidence intervals on F1 are computed using 2,000 plan-level resamples, resampling complete plans rather than individual action pairs to account for within-plan dependence.

Code

All code can be found at the following link: https://github.com/markmartirosian1/LinearProbingProceduralPlans/tree/main/dag-probing